

Fitness Landscape Compression for Genetic Programming

ZHIXING HUANG, School of Computer Science and Engineering, South China University of Technology, China; Centre for Data Science and Artificial Intelligence & School of Engineering and Computer Science, Victoria University of Wellington, New Zealand

BING XUE, YI MEI, and FANGFANG ZHANG, Centre for Data Science and Artificial Intelligence & School of Engineering and Computer Science, Victoria University of Wellington, New Zealand

WOLFGANG BANZHAF, Department of Computer Science and Engineering, BEACON Center for the Study of Evolution in Action, and Ecology, Evolution and Behavior Program, Michigan State University, USA

MENGJIE ZHANG, Centre for Data Science and Artificial Intelligence & School of Engineering and Computer Science, Victoria University of Wellington, New Zealand

Searching for programs by genetic programming has been successfully applied to many areas. However, the vast and rugged fitness landscape of searching programs often limits the further improvement of genetic programming. It takes a significant amount of time for genetic programming to traverse the huge search space and escape from local optima. To improve the learning effectiveness and efficiency of genetic programming, this paper explicitly compresses fitness landscapes. Specifically, we prioritize the primitives of genetic programming by a fitness landscape optimization method and maintain a dynamic and limited set of useful primitives that form the compressed landscape. We implement the new method with linear genetic programming. We verify the new genetic programming method on two types of problems, including symbolic regression and dynamic combinatorial optimization problems. Our results show that the proposed method has a very competitive learning performance with state-of-the-art methods, with a very promising training efficiency for both supervised and learn-to-optimize tasks. We further analyze and visualize example compressed landscapes, verifying the effectiveness of the proposed landscape compression method.

CCS Concepts: • **Computing methodologies** → **Discrete space search**.

Additional Key Words and Phrases: Genetic Programming, Fitness Landscape, Symbolic Regression, Learn-to-optimize, Dynamic Job Shop Scheduling.

1 Introduction

Genetic programming (GP) is a meta-heuristic method that is used to search for computer programs, such as mathematic expressions and decision rules [De Franca et al. 2025; Koza 1995]. GP stochastically searches for

Authors' Contact Information: Zhixing Huang, zhixinghuang@scut.edu.cn, School of Computer Science and Engineering, South China University of Technology, Guangzhou, China; and Centre for Data Science and Artificial Intelligence & School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand; Bing Xue, bing.xue@ecs.vuw.ac.nz; Yi Mei, yi.mei@ecs.vuw.ac.nz; Fangfang Zhang, fangfang.zhang@ecs.vuw.ac.nz, Centre for Data Science and Artificial Intelligence & School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand; Wolfgang Banzhaf, banzhafw@msu.edu, Department of Computer Science and Engineering, BEACON Center for the Study of Evolution in Action, and Ecology, Evolution and Behavior Program, Michigan State University, East Lansing, USA; Mengjie Zhang, mengjie.zhang@ecs.vuw.ac.nz, Centre for Data Science and Artificial Intelligence & School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2688-3007/2026/9-ART

<https://doi.org/10.1145/3847110>

computer programs and explores new solutions based on genetic operators. Compared with other computer program search methods, GP has the advantages of variable solution size, easy integration of domain knowledge, and producing compact and interpretable solutions. GP helps humans explore unknown models and design functions based on customized primitives, which is very useful for a wide range of applications such as chip design [Shi et al. 2024], decision rule design [Zhang et al. 2024], and interpreting machine learning models [Mei et al. 2023].

All possible computer programs, together with their neighborhood structure and objective values, consist of a fitness landscape (FL) for GP methods. FL is a widely adopted concept in optimization that comes from evolutionary biology. It comprises a search space (all possible solutions), a neighborhood structure, and a fitness function (i.e., objective function). Searching for computer programs is essentially to wander along the FL to find the best-fit solutions. However, because of the exponentially increasing number of possible solutions and the diverse physical meanings of GP primitives, the FL of a GP method is usually vast, irregular, and highly rugged, which limits further improvement of GP.

To efficiently search for effective solutions, we look forward to retaining effective symbols, which are likely to form optimal solutions, in a smaller search space. We call this process of retaining important parts of fitness landscapes as fitness landscape compression.

Compressing the FL for GP is non-trivial and substantially different from the compression in numerical search spaces (e.g., dimensional reduction). First of all, the ultimate goal of compressing fitness landscapes for GP is to reduce the search space as much as possible while keeping the (near) optimal solutions in the compressed search space. It is essentially a lossy compression. This compares to dimension reduction in numerical search spaces, which reduces dimensions as much as possible while retaining as much information of the original space as possible. Second, the FL of GP is discrete and irregular because of the symbolic nature of its elements and the complicated neighborhood structure, while numerical search spaces are usually continuous and have straightforward neighborhood structures (e.g., Euclidean distance).

There have been some related GP studies on this topic. For example, feature selection for GP tries to highlight certain search regions based on the heuristic methods (e.g., permutation importance and primitive frequency) [Chen et al. 2017; Zhang et al. 2021a]. Grammar-guided GP adds constraints to reduce the search space by grammar rules [Fonseca and Poças 2023; Huang et al. 2024b]. However, these methods are not explicitly designed for compressing FLs. They often fail to compress fitness landscapes efficiently and require a lot of domain knowledge in designing grammar rules. Some other GP methods are coming with similar ideas of compressing symbols, such as automatically defined functions [Koza 1994] and encapsulating program outputs into an input terminal [Reyes Fernández De Bulnes et al. 2024]. However, they did not compress the fitness landscape but expanded the primitive set with the newly compressed symbol.

This paper proposes a fitness landscape compression (FLC) method for GP with three main steps. First, we integrate multiple low-level primitives into one higher-level primitive. Second, we optimize the neighborhood structure of fitness landscapes to prioritize the most likely useful primitives and to form a smooth landscape. Third, we reduce less promising primitives to compress landscapes. We implement a new GP method based on FLC. Specifically, this paper focuses on linear genetic programming (LGP) [Brameier and Banzhaf 2007] as it is easy for LGP to define its fitness landscape. We denote LGP with fitness landscape compression as **Landscape-Compressed Linear Genetic Programming (LCLGP)**. This paper is an extension from the existing work of fitness landscape optimization [Huang et al. 2025].

This paper has three main contributions.

- (1) We explicitly address the FLC for LGP methods. The compressed FL helps LGP focus on a reduced and higher-level search space and improves LGP search effectiveness and efficiency. The compressed

landscape also provides a potential perspective to automatically reduce the search space and a more concise perspective for analyzing FLs than the entire landscape.

- (2) We verify the effective compression of FLs via three hypotheses. The verification of the three hypotheses shows that the proposed method successfully compresses the FLs and aggregates effective primitives.
- (3) The applications on both supervised learning and learn-to-optimize tasks (e.g., symbolic regression and dynamic combinatorial optimization problems) shows that compressed FLs significantly improve LGP performance and save substantial training time.

2 Preliminary

2.1 Fitness Landscapes of Searching for Computer Programs

Searching for computer programs is an abstract problem, encompassing a wide range of applications, such as symbolic regression, program synthesis, and decision rule designs. It is expressed as an optimization problem to find the best computer program ρ^* :

$$\rho^* = \arg \min_{\rho \in \mathcal{P}} f(\rho),$$

where ρ is a computer program from a search space $\mathcal{P}$, and $f(\cdot) : \mathcal{P} \mapsto \mathbb{R}$ is a fitness function of a specific task. In this paper, we focus on minimization problems. $\mathcal{P}$ is composed of a set of given primitives $\mathcal{I} = \{\sigma\}$ and their grammar rules $\mathcal{T}$, i.e., $\mathcal{P} = \{\mathcal{I}, \mathcal{T}\}^L$ where L is the maximum number of primitives in ρ . All the possible ρ are combinations of σ , i.e., $\rho = [\sigma_L \circ \sigma_{L-1} \circ \dots \circ \sigma_1]$, and $\circ$ denotes the combination based on the grammar rules $\mathcal{T}$. When we synthesize single-output mapping functions, ρ is expressed as a function $\rho(\cdot) : \mathbb{R}^m \mapsto \mathbb{R}$ where m is the number of input features.

Search methods explore $\mathcal{P}$ based on the neighborhood structure $v(\cdot)$. $v(\mathcal{P}|\rho)$ defines which solutions in $\mathcal{P}$ are accessible from a given solution ρ . Particularly, in GP, v is usually equivalent to genetic operators because of the symbolic nature of GP elements. In this paper, we assume that all solutions in $\mathcal{P}$ are accessible to each other via the neighborhood structure through one or multiple steps. The search space $\mathcal{P}$, the neighborhood structure v , and the fitness function f define the fitness landscape $\mathcal{L}$ of a computer program search problem, i.e., $\mathcal{L} = \{f, \mathcal{P}, v\}$. Note that FL is different from loss surfaces, which is another similar term in machine learning community. An FL focuses on analyzing the entire search space, while a loss surface mainly describes the performance improvement of a machine learning model during training.

Improving the search effectiveness of GP can be formalized as minimizing the fitness expectation of sampling ρ through a search method θ in $\mathcal{L}$ (for minimization problems):

$$\min_{\theta(\mathcal{L})} \mathbb{E}[f(\rho)|\rho \sim \theta(\mathcal{L})]. \quad (1)$$

In this work, we primarily improve the search effectiveness by manipulating $\mathcal{L}$, rather than refining search methods θ (e.g., adjusting operator rates, step size, or primitive sampling distribution). We construct a compressed $\mathcal{L}$ from the problem-defined one, and apply the same search method in it.

2.2 Linear Genetic Programming

LGP [Brameier and Banzhaf 2007] is a popular GP variant with a linear representation. Specifically, each LGP individual is a variable-length sequence of register-based instructions. An LGP individual represents a computer program by sequentially executing its instructions with output of the final result from defined output registers. Fig. 1 shows an example LGP program. The linear representation leads to genetic operators different from tree-based structures, such as linear crossover, macro and micro mutation. Nowadays, LGP has shown its theoretical and practical superiority over a wide range of applications, such as multiclass classification [Downey et al. 2010],

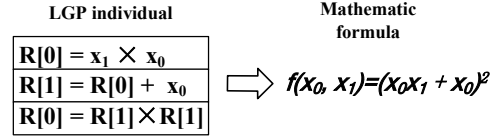


Fig. 1. An example LGP individual with three instructions.

mechanical control [Passaggia et al. 2022; Praczyk and Szymkowiak 2023], dynamic combinatorial optimization [Huang et al. 2024a,b], and search space and landscape analysis [Hu and Banzhaf 2018; Hu et al. 2013, 2011].

The linear representation has three advantages in studying the fitness landscape of GP. First, an LGP individual can be easily transformed into a numerical vector if we treat each instruction as a symbol and give each symbol an index. This facilitates our modeling on FLs. Second, defining the neighborhood structure of LGP individuals is straightforward based on the instruction editing distance, e.g., adding or removing one instructions. Third, LGP instructions are flexible to represent higher-level symbols by including multiple functions within one instruction.

2.3 Related Work

2.3.1 Fitness Landscape in GP. The FL is an important perspective for understanding GP search. A smoother FL with fewer local optima implies an easier search. Most existing studies on FL in GP focus on analyzing the hardness of the FL. For example, Vanneschi et al. [Vanneschi 2004; Vanneschi and Tomassini 2003; Vanneschi et al. 2003] and Ćorić et al. [Ćorić et al. 2021] thoroughly examined using fitness distance correlation to analyze the problem difficulty in GP. The fitness distance correlation estimates the correlation between the distance and the fitness difference from the global optimum, in which a larger correlation implies an easier FL [Jones and Forrest 1995]. Vanneschi et al. [Vanneschi et al. 2004, 2006] further proposed to use a negative scope coefficient to measure the difficulty of GP search. The negative scope coefficient measures how likely GP moves from a good solution to a poor one, also known as bad evolvability. In addition, local optima network [Durasevic et al. 2020; He and Neri 2023], random-walk sampling methods [Kinnear 1994; Slaný and Sekanina 2007], and other LGP-based FL analyses [Hu and Banzhaf 2018; Hu et al. 2013, 2011, 2012] have been used in existing GP studies. However, only few studies tried manipulating FLs in GP to simplify the search.

2.3.2 FL Manipulation Methods. The goal of compressing FLs is to minimize the size of the search space $\mathcal{P}$ while maximizing the likelihood that the reduced space contains the solution(s) with (near) optimal fitness. Some similar ideas can be seen in numerical optimization studies. For example, Nobile et al. [Nobile et al. 2019] and Papetti et al. [Papetti et al. 2021, 2023] proposed to use dilation functions to transform the original search space. The dilation function smoothes FLs by expanding promising search regions and shrinking the search regions with poor fitness. Lyu et al. [Lyu et al. 2021] proposed a neural-network-based surrogate model to smooth the local optima and disclose the global trend on FL. However, existing methods for numerical optimization require a lot of problem-specific knowledge in designing smoother FLs and are hard to extend to computer program search problems.

Some existing GP studies intrinsically show an idea of manipulating FLs, but not compressing FLs. Evolving automatically defined functions (ADFs) is a typical method for manipulating FLs [Fernandes et al. 2023; Koza 1994; Ma et al. 2025]. By constructing higher-level functions, GP with ADFs replaces these useful higher-level functions (i.e., building blocks) with a symbol, saving GP search effort in re-constructing these building blocks. However, rather than reducing the search space, GP methods with ADFs often expand the search space because they simply include ADFs as additional primitives in the primitive set. Expanding the search space might reduce the probability of finding effective solutions. Similar to ADFs, other modular GP methods can be treated as a related

method that manipulates FLs by decomposing the original search space into smaller ones [Rodriguez-Coayahuitl et al. 2025]. However, the search space decomposition might be dependent on the domain knowledge and is fixed during GP search. In addition to ADFs and modular GPs, existing studies also improve primitive sets by empirically testing possible functions [Nicolau and Agapitos 2021; Wang and Soule 2004] or by transfer learning from other related problems [Helmuth et al. 2020; Muñoz et al. 2020]. Although these improvement methods can identify and construct effective primitives for GP in certain problems, they are often specific to certain domains and rely on careful manual designs.

Multitask and multiform optimization are emerging topics that harnesses similar FLs to enhance search performance [Huang et al. 2024a; Jiao et al. 2023, 2024; Zhang et al. 2025; Zhong et al. 2025]. However, selecting and constructing these similar FLs for unseen problems are non-trivial and problem-specific. Searching in those related but not necessarily the same FLs might waste computation resources as well.

Grammar-guided GP and strongly typed GP are special GP variants that reduce the search space based on grammar rules or data types given beforehand [Fonseca and Poças 2023; Huang et al. 2024b; Whigham 1995]. However, manually reducing the search space is not flexible and ineffective in many cases because designing grammar rules and data types requires a lot of expertise and human labor.

Recently, Huang et al. [Huang et al. 2025] proposed a fitness landscape optimization (FLO) method that automatically reduces local optima. The main idea of FLO is to index LGP instructions and optimize the neighborhood of LGP solutions by changing the instruction indices. The local optima on an FL would be reduced by aggregating good solutions and separating good and poor solutions. It has shown superior performance in reducing the hardness of FLs over frequency-based methods and some advanced methods.

In this paper, we substantially extend FLO to FL compression from three aspects:

- (1) We construct a dynamic set of higher-level instructions based on the idea of modular GP during evolution. By contrast, the set of instructions of FLO is static and has to enumerate all possible instructions, which is very time-consuming and infeasible for high-dimensional problems.
- (2) We updated the landscape optimization objectives and their gradient based on the dynamic set of instructions. Furthermore, we added a new landscape optimization objective, the L2-norm of instruction indices of good solutions, to encourage the indices of effective solutions to gather around the origin. Thereby, we can apply OAM to efficiently traverse the landscape.
- (3) We perform a more comprehensive verification on the proposed method than FLO, especially in high-dimensional problems, benefit from the much more efficient training than FLO.

2.3.3 Differences From Designing Better Search Methods. To clarify the newly proposed fitness landscape compression, we discuss the difference between FLC (i.e., optimizing $\mathcal{L}$ in Eq. (1)) and designing better search methods (i.e., improving $\theta(\mathcal{L})$). The key difference is that FLC reduces the set of neighbor solutions, while a search method decides how to sample from the set of neighbor solutions within limited resources. Regarding that the neighborhood structures in GP studies are usually equivalent to the genetic operators, the set of solutions produced by the search method given a specific solution will end up with the set of neighbor solutions with infinite resources. The following are three representative search methods for improving GP effectiveness.

Highlighting useful primitives based on their frequency in GP population is a simple but effective search method [Al-Helali et al. 2021; Huang et al. 2020; Zhang et al. 2021b]. These studies assume that a primitive with high frequency is useful. They improve GP effectiveness by adjusting the feature distribution when sampling new solutions.

Semantic GP is an effective search method based on the behaviors (i.e., input-output maps) of primitives [Huang et al. 2022a; Moraglio et al. 2012; Vanneschi et al. 2025]. Semantic GP prefers neighbor solutions with a smaller semantic gap to a desired semantics. Although semantic GP, such as geometric semantic GP [Moraglio

et al. 2012], theoretically shows that the fitness landscape over the semantic space is a cone-like landscape, they do not manipulate the actual search space and cannot guarantee a program with the specific semantics in practice.

Archive-based GP is a common search method to enhance population diversity [Hong et al. 2024; Xu et al. 2021]. Archive-based GP remembers elite individuals from previous generations through an archive. Unlike reducing search spaces to retain promising symbols, archive-based GP provides more diverse search directions for each move, particularly for recombination operators.

3 Fitness Landscape Compression and Exploitation

3.1 Main Idea

Based on Eq. (1), fitness landscape compression is essentially minimizing the fitness expectation by optimizing the search space $\mathcal{P}$. We optimize the search space by improving the sampling probability ($\frac{\|\mathcal{P}_{\theta(f(\rho))}\|}{\|\mathcal{P}_{\theta}\|}$) of solutions with good fitness, as shown in Eq. (2).

$$\min_{\theta(\mathcal{L})} \mathbb{E}[f(\rho)|\rho \sim \theta(\mathcal{L})] = \min_{\theta(\mathcal{L})} \sum_{\rho \sim \theta(\mathcal{L})} f(\rho) \Pr(f(\rho)) \Rightarrow \min_{\mathcal{P}_{\theta}} \sum_{\mathcal{P}_{\theta(f(\rho))} \subseteq \theta(\mathcal{P})} f(\rho) \frac{\|\mathcal{P}_{\theta(f(\rho))}\|}{\|\mathcal{P}_{\theta}\|}, \quad (2)$$

where $\mathcal{P}_{\theta}$ indicates the set of solutions sampled by θ , $\mathcal{P}_{\theta(f(\rho))}$ indicates the solution set with the fitness of $f(\rho)$ sampled through the search method θ , and $\|\cdot\|$ indicates the cardinality of a solution set. Specifically, we fix the f and v in $\mathcal{L}$ and optimize $\mathcal{P}$ by 1) increasing $\|\mathcal{P}_{\theta(f(\rho))}\|$ by including more effective high-level symbols into the symbol set $\mathcal{J}$ and 2) reducing $\|\mathcal{P}_{\theta}\|$ by reducing $\|\mathcal{J}\|$. Thus, compressing FL is essentially an optimization on $\mathcal{J}$. To simplify our optimization, we do not consecutively reduce $\|\mathcal{J}\|$ over generations, but set a fixed maximum size K of $\mathcal{J}$ and include as many effective symbols as possible.

Fig. 2 shows the schematic diagram of FLC. There are three main steps in optimizing $\mathcal{J}$. We first collect effective symbols from our sampled solutions and add them into $\mathcal{J}$. $\mathcal{J}$ includes the symbols and their corresponding indices. The indices essentially define the FL. Second, we prioritize symbols by optimizing their indices. Third, we remove less effective symbols from $\mathcal{J}$ until $\|\mathcal{J}\| \leq K$. We expect that the compressed FL formed by the updated $\mathcal{J}'$ is small and covers the (near) optimal solutions. Therefore, searching for effective solutions on the compressed FL is more efficient than on the raw search space. By optimizing the compressed FL, we improve the proportion of better solutions in $\mathcal{P}$ and thus minimize the fitness expectation.

Notably, the compressed FL might be deceptive and might not include the right solutions, especially at the early stage of the evolution. To overcome the deceptive FL, we still produce offspring by basic mutation operators, which help explore unknown solutions from the entire search space. We also update the FL periodically to keep the effective instructions up-to-date.

3.2 Evolutionary Framework

We propose a new LGP method based on fitness landscape compression, denoted as LCLGP¹. Specifically, we treat each LGP instruction as a symbol and $\mathcal{J}$ as an instruction set. Each instruction has an index indicating its location on the FL. An LGP solution is equivalent to an index vector by converting every instruction to the index. The Euclidean distance of the index vectors defines the neighborhood structures of LGP solutions on FLs. Although the arbitrary initial indices of instructions might introduce biases on the neighborhood of LGP solutions, the neighborhood will be optimized by the fitness landscape optimization component during evolution.

¹The source code of LCLGP can be accessed from 1) Java: <https://github.com/Zhixing1020/Linear-Genetic-Programming-LGP-and-Applications/tree/main/src/zhixing/cpxInd/algorithm/LandscapeOptimization/FLReductionLGP>. 2) Python: https://github.com/Zhixing1020/LGP4PY_public/tree/main/tasks/symbreg/algorithm/LandscapeOptimization

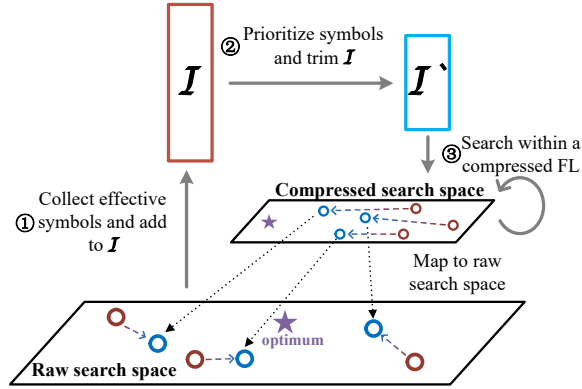


Fig. 2. Schematic diagram of FLC. Red nodes on the search space indicate the sampled solutions, and the blue nodes are the newly-to-be-sampled nodes. The purple star is the optimal solution.

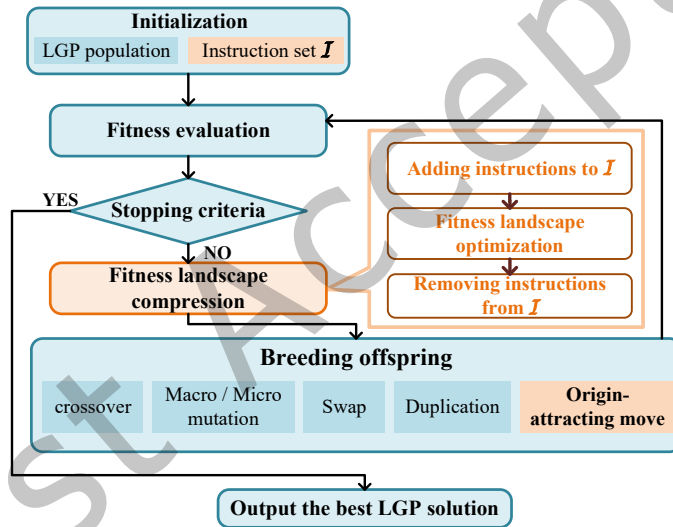


Fig. 3. Evolution framework of LCLGP. The new components are highlighted in orange.

We integrate the three steps of FLC into the evolutionary framework of LCLGP, as shown in Fig. 3. LCLGP first randomly initializes the LGP population and a reduced instruction set $\mathcal{I}$. After evaluating the fitness of individuals, LCLGP performs FLC to update the instruction set $\mathcal{I}$. Then, LCLGP produces new LGP solutions through a set of genetic operators, including linear crossover, macro and micro mutation, swapping consecutive instructions, duplicating competitive individuals, and a newly proposed landscape-based genetic operator, origin-attracting move (OAM). OAM produces offspring from the compressed FL, while the other operators produce from the raw FL. LCLGP iteratively performs FLC and explores the raw FL by the basic operators to improve the global search capability. After iterating a given number of generations, we output the best-of-the-run LGP solution.

3.3 Individual Representation

To include more effective instructions in $\mathcal{J}$, LGP explores higher-level instructions. Given the linear representation, each LGP instruction consists of one to multiple functions, leading to high-level instructions. A high-level instruction accepts the values from registers, executes all the functions in the instruction, and stores the final calculation results to a destination register. For example, the first two instructions can be represented by the third multi-function instruction.

Basic instructions: $R0 = x + y; R0 = R0 \times x;$

High-level instruction: $R0 = x \times (x + y);$

Instructions with multiple functions allow LGP solutions and the instruction set to cover a wider range of basic symbols (e.g., instructions with a single function) with a limited size.

3.4 Adding Instructions

LGP periodically adds instructions from elite and competitive solutions into $\mathcal{J}$. Specifically, we check the instructions used by the elite and competitive solutions in the population and add them into $\mathcal{J}$ if they are not in $\mathcal{J}$. We select the competitive solutions by a tournament selection to improve the diversity. The index of a newly added instruction is set as the $\max \sigma_{\mathcal{J}} + 1$ where $\sigma_{\mathcal{J}}$ is the indices in $\mathcal{J}$. We also summarize the instruction frequency $q(\sigma)$ in the LGP population for each instruction $\sigma \in \mathcal{J}$ when adding instructions.

3.5 Fitness Landscape Optimization

We perform the optimization of the fitness landscape to prioritize the instructions in $\mathcal{J}$ based on [Huang et al. 2025]. The main idea of the fitness landscape optimization is to adjust the instruction indices in $\mathcal{J}$ to optimize the fitness landscape objectives $F(\mathcal{J})$. It has been shown that the fitness landscape optimization method improves the smoothness of the FL (i.e., a high correlation between the fitness discrepancy and the search space distance).

The optimization over instruction indices results in different neighborhoods on the fitness landscape. The neighborhood of each solution is defined by a neighborhood structure of Euclidean distance over index vectors, i.e., $\nu(\rho) = \{\rho' \mid \|\rho_{\mathcal{J}} - \rho'_{\mathcal{J}}\|_2 \leq \epsilon\}$. $\rho_{\mathcal{J}}$ and $\rho'_{\mathcal{J}}$ here are the index vectors of ρ and ρ' , respectively, and ϵ is a predefined neighborhood threshold.

Fig. 4 shows an illustrative example of fitness landscape optimization reproduced from [Huang et al. 2025]. The fitness landscape optimization in Fig. 4 tries to aggregate the good solutions and separate them from poor solutions to smoothen the landscape. Suppose there are twelve possible instructions, **A** to **L**, each with an index specified by $\mathcal{J} = [0, 1, \dots, 11]^T$. Each instruction makes up a program in the search space, e.g., $G_1 = [\mathbf{A}] \Rightarrow [0]$, $G_2 = [\mathbf{B}] \Rightarrow [1]$. The instruction indices determine the position and neighbors of each program. We can see that the original $\mathcal{J}$ leads to a rugged landscape, where there are two local optimum (minimum), G_4 and G_7 . To smoothen the landscape, fitness landscape optimization aggregates good programs (e.g., G_1 and G_{12}) and separate good and poor programs (e.g., G_1 and G_6) in the landscape by changing the instruction indices, e.g., **A** with index 0 and **L** with index 1. We can see that the optimized landscape is smoother than before, and has less local optima.

There are two main steps for fitness landscape optimization:

- (1) We collect solutions from the current population based on the optimization objective $F(\mathcal{J})$.
- (2) Iteratively update $\mathcal{J}$ to minimize $F(\mathcal{J})$ by a stochastic gradient search method. The new instruction indices are updated by Eq. (5).

3.5.1 Optimization Objectives. As we are optimizing a compressed FL that includes as many effective instructions as possible, we minimize the distance between good solutions on the compressed FL ($D(\mathcal{J})$, also named as inner distance in [Huang et al. 2025]) and the L2-norm of instruction indices of good solutions ($L_2(\mathcal{J})$). The L2-norm

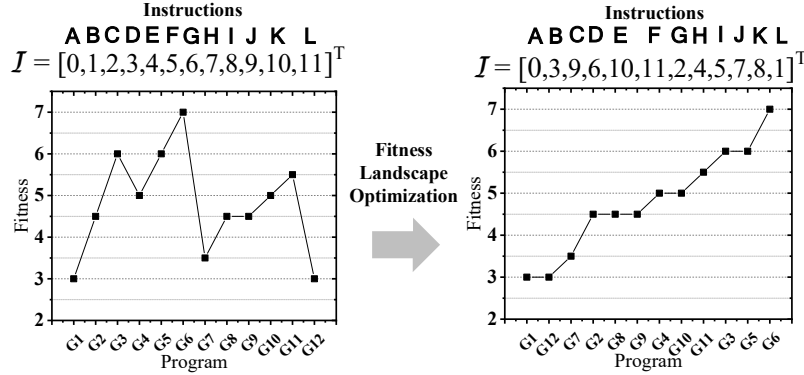


Fig. 4. An example of fitness landscape optimization over 1-instruction programs [Huang et al. 2025].

objective encourages the fitness landscape optimization method to prioritize effective instructions in an ascending order. The optimization objective is formulated as Eq. (3).

$$F(\mathcal{J}) = \frac{\alpha_1}{D(\mathcal{J}_0)} D(\mathcal{J}) + \frac{\alpha_2}{L_2(\mathcal{J}_0)} L_2(\mathcal{J}), \quad (3)$$

where

$$D(\mathcal{J}) = \frac{1}{|\mathcal{B}|^2} \sum_{a \in \mathcal{B}} \sum_{b \in \mathcal{B}, a \neq b} \sum_{l=1}^L (\sigma_{(a,l),\mathcal{J}} - \sigma_{(b,l),\mathcal{J}})^2,$$

$$L_2(\mathcal{J}) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \|\rho_{i,\mathcal{J}}\|_2^2 = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(\sum_{l=1}^{L_i} \sigma_{l,\mathcal{J}}^2 \right).$$

Here, we square the L-2 norm of instruction indices to facilitate the gradient calculation. $\sigma_{l,\mathcal{J}}$ indicates the index of instruction σ_l . $\mathcal{B}$ is the set of elite solutions. α_1 and α_2 are the coefficients of the two sub-objectives, and $\mathcal{J}_0$ is the original instruction set for normalizing objective values.

Notably, we did not consider the semantic information of instructions in the FL compression. Semantic information is an effective perspective to define the neighborhood of GP programs. However, the existing study [Huang et al. 2025] has shown that optimizing fitness landscapes based on semantic information performs similarly to based on fitness values. Given that manipulating semantics often introduces computation overhead, we decided not to consider semantic information in the current design.

3.5.2 Stochastic Gradient Search. We apply a stochastic gradient descent method to optimize $F(\mathcal{J})$ [Huang et al. 2025]. The gradient of $F(\mathcal{J})$ is shown as Eq. (4).

$$\frac{\partial F}{\partial \mathcal{J}} = \frac{\alpha_1}{D(\mathcal{J}_0)} \frac{\partial D}{\partial \mathcal{J}} + \frac{\alpha_2}{L_2(\mathcal{J}_0)} \frac{\partial L_2}{\partial \mathcal{J}}, \quad (4)$$

where

$$\frac{\partial D}{\partial \mathcal{J}} = \left[\frac{\partial D}{\partial \sigma_{\mathcal{J},1}}, \dots, \frac{\partial D}{\partial \sigma_{\mathcal{J},k}}, \dots, \frac{\partial D}{\partial \sigma_{\mathcal{J},K}} \right]^T,$$

$$\begin{aligned}\frac{\partial D}{\partial \sigma_{\mathcal{J},k}} &= \frac{1}{|\mathbb{B}|^2} \sum_{a \in \mathbb{B}} \sum_{b \in \mathbb{B}, a \neq b} \sum_l^L \frac{(\sigma_{(a,l),\mathcal{J}} - \sigma_{(b,l),\mathcal{J}})^2}{\partial \sigma_{\mathcal{J},k}}, \\ \frac{\partial L_2}{\partial \mathcal{J}} &= \left[\frac{\partial L_2}{\partial \sigma_{\mathcal{J},1}}, \dots, \frac{\partial L_2}{\partial \sigma_{\mathcal{J},k}}, \dots, \frac{\partial L_2}{\partial \sigma_{\mathcal{J},K}} \right]^T, \\ \frac{\partial L_2}{\partial \sigma_{\mathcal{J},k}} &= \frac{1}{|\mathbb{B}|} \sum_{i \in \mathbb{B}} \left(\sum_{l=1}^{L_i} \frac{\partial \sigma_{i,l,\mathcal{J}}^2}{\partial \sigma_{\mathcal{J},k}} \right).\end{aligned}$$

$\sigma_{l,\mathcal{J}} = \sigma_{\mathcal{J},K}$ when they are indicating the same instruction. The new indices $\mathcal{J}'$ are updated by Eq. (5) iteratively.

$$\mathcal{J}' = \lfloor \mathcal{J} - \text{sign}\left(\frac{\partial F}{\partial \mathcal{J}}\right) - U(\alpha K) \cdot \frac{\partial F}{\partial \mathcal{J}} \rfloor, \quad (5)$$

where $U(\alpha K)$ returns a uniform random value in $[0, \alpha K]$. α is a predefined step size parameter ranging from $(0, 1]$. The floor operation maps the continuous values to integers (i.e., indices). By iteratively updating the instruction indices, we expect the indices of effective instructions aggregate to small index values.

3.6 Removing Instructions

After adding instructions and prioritizing them, we likely have a large instruction set, $\|\mathcal{J}\| > K$. To maintain a short $\mathcal{J}$, we remove instructions from $\mathcal{J}$ based on their indices. Since we aggregate effective instructions to small index values in fitness landscape optimization, a large instruction index implies a large removing probability. Specifically, we remove an instruction σ_k if

$$0.1 + 0.9 \frac{\sigma_{\mathcal{J},k}}{K} \geq u \sim U(0, 1),$$

where $U(0, 1)$ is the uniform distribution over $[0, 1]$. The removing probability implies that every instruction in $\mathcal{J}$ will be removed with at least a probability of 0.1, which help improve the diversity of $\mathcal{J}$. We remove instructions from $\mathcal{J}$ until $\|\mathcal{J}\| \leq K$. Because we retain the most prioritized instructions in $\mathcal{J}$ after each optimization, the FL formed by the reduced $\mathcal{J}$ is equivalent to a compressed FL.

3.7 Origin-attracting Move

In the step of fitness landscape optimization, we prioritize effective instructions by small index values. To this end, we prefer small index values for the newly sampled instructions. Therefore, we propose the origin-attracting move to produce new LGP solutions. OAM treats an LGP solution as a point (i.e., an integer vector) on the compressed FL. OAM moves the LGP solution to a new point by manipulating instruction indices. As every index stands for an instruction, the new point on the FL stands for a new LGP solution.

Fig. 5 shows a simple example of OAM in the optimized FL in Fig. 4. Suppose we have a program $G_4 = [6]$, then OAM produces a new program from G_4 by moving it toward the origin. In other word, OAM reduces the index of 6 into a smaller value, such as 2, and gets the new program G_7 . As the programs closer to the origin are more likely to have better fitness in the optimized FL, OAM is effective in producing better programs.

Alg. 1 is the pseudo code of OAM. We first select the to-be-moved instructions $\{\sigma\}$ and transform them into indices $\{\sigma_{\mathcal{J}}\}$. When the to-be-moved instructions are not in $\mathcal{J}$, we add this instruction into $\mathcal{J}$ with the largest index. For each instruction index in $\{\sigma_{\mathcal{J}}\}$, we move it toward the origin based on

$$\sigma'_{\mathcal{J},k} \leftarrow \lfloor \sigma_{\mathcal{J},k} - \max(1, U(0, \sigma_{\mathcal{J},k} V)) \rfloor,$$

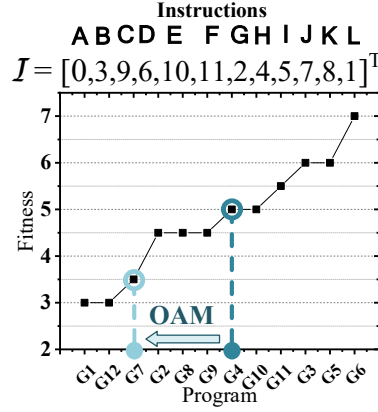


Fig. 5. An illustrative example of OAM.

where V decides the maximum moving distance (i.e., the percentage toward the origin). If $\sigma'_{j,k}$ is out of the range of instruction indices in $\mathcal{I}$, we randomly select an instruction from $\mathcal{I}$ as $\sigma'_{j,k}$. We then convert the new index $\sigma'_{j,k}$ into the instruction σ'_k and put it into its corresponding place in ρ' . Notably, to improve the diversity of sampled instruction, we perform a tabu search based on instruction frequency $q(\sigma'_k)$. If instruction σ'_k has been frequently used in the population, we apply a micro mutation on σ'_k before putting it into ρ' . After that, we increase or reduce the program size of ρ' based on an instruction adding rate p_{add} , and make sure that at least one newly update instruction is effective by manipulating their destination register.

4 Experiment Design

4.1 Test Problems

We verify the effectiveness of LCLGP by two applications, symbolic regression and decision rule design. First, we verify the effectiveness of the proposed FLC method on common symbolic regression (SR) and dynamic combinatorial optimization benchmark problems. Second, we extend LCLGP to work with a linear regression model and compare with a wide range of state-of-the-art methods on SRBench [De Franca et al. 2025].

We select seven SR problems in our first experiment. They are Nguyen4, Nguyen5, Keijzer11, R1, Airfoil, BHouse, and Redwine. The first four problems are synthetic benchmarks whose settings follow those in [Huang et al. 2025, 2022a]. The last three problems are real-world problems from UCI machine learning datasets with unknown ground truths. Relative square error (RSE) is the fitness function in our SR experiments.

For the dynamic combinatorial optimization problem, we verify FLC on dynamic job shop scheduling (DJSS) problems. DJSS is a typical dynamic combinatorial optimization problem and is the foundation of many other problems, such as dynamic flexible job shop scheduling and workflow scheduling. Here, we select six DJSS scenarios for verification, each with a different optimization objective. The objectives include maximum tardiness (Tmax), mean tardiness (Tmean), weighted mean tardiness (WTmean), maximum flowtime (Fmax), mean flowtime (Fmean), and weighted mean flowtime (WFmean). All six scenarios simulate DJSS problems with a utilization level of 0.95. The settings of the DJSS simulation follow the ones in [Huang et al. 2025]. Each SR dataset and DJSS scenario is simulated by 50 independent runs with different random seeds.

Algorithm 1: Origin-attracting Move

Input: An LGP solution ρ , an instruction set $\mathcal{I}$, the maximum number of varying instructions M , the maximum moving distance V , and an instruction adding rate p_{add} .

Output: A new LGP solution ρ'

```

1  $m \leftarrow U(0, \max(\|\rho\|, M))$ ,  $\rho' \leftarrow \rho$ .
2  $\{\sigma\} \leftarrow$  randomly select  $m$  instructions from  $\rho$ .
3  $\{\sigma_j\} \leftarrow$  convert  $\{\sigma\}$  into corresponding indices based on  $\mathcal{I}$ .
  // For each chosen (to-be-mutated) instruction, do
4 for  $\sigma_{j,k} \in \{\sigma_j\}$  do
  // Moving the instruction index to the origin.
5    $\sigma'_{j,k} \leftarrow \lfloor \sigma_{j,k} - \max(1, U(0, \sigma_{j,k}V)) \rfloor$ .
6   if  $\sigma'_{j,k}$  is out of the range of indices then
7      $\sigma'_{j,k} \leftarrow$  randomly select an instruction from  $\mathcal{I}$ .
8   Get instruction  $\sigma'_k$  based on index  $\sigma'_{j,k}$ .
  // Slightly change the new instruction  $\sigma'_k$  if it has been frequently used in the population.
9   if  $U(0, 1) < \min(0.95, 2q(\sigma'_k)/\text{population size})$  then
10     $\sigma'_k \leftarrow \text{micro\_mutation}(\sigma'_k)$ 
  //  $\sigma'_k$  supersedes  $\sigma_k$ .
11  Set the corresponding instruction  $\sigma_k$  in  $\rho'$  as  $\sigma'_k$ .
  // adding instructions.
12 if  $U(0, 1) < p_{add}$  and  $\|\rho'\|$  has not reached maximum program size then
13   Randomly select an instruction from  $\mathcal{I}$  and add into  $\rho'$ .
  // removing instructions.
14 else if  $\|\rho'\|$  has not reached minimum program size then
15   Randomly remove an instruction from  $\rho'$ .
16 Make sure at least one newly updated instruction in  $\rho'$  is effective.
17 Return  $\rho'$ ;

```

SRBench is a commonly used benchmark for symbolic regression [De Franca et al. 2025]. There have been a large number of advanced SR methods in SRBench, each with test R-square values, program sizes, and training times, providing a comprehensive perspective for analyzing SR methods. Here, we focus on the black-box regression problems in SRBench, covering 122 real-world datasets.

4.2 Compared Methods

We select a basic LGP method and the LGP with fitness landscape optimization method (FLO) [Huang et al. 2025] as the compared methods in the first experiment. The basic LGP method serves as a baseline, and FLO is one of the state-of-the-art methods for the both SR and DJSS problems. The settings of the both basic LGP and FLO methods follow [Huang et al. 2025]. Specifically, the LGP population evolves with 256 individuals for 200 generations. LGP individuals have up to 100 instructions for SR problems and up to 50 instructions for DJSS problems.

LCLGP has its own specific parameters, while the others follow the parameter settings of the two compared methods. First, each LCLGP instruction has up to 3 functions to construct a high-level instruction. To ensure a fair setting on the maximum program size, LCLGP individuals have up to 30 instructions for SR problems and up to 20 instructions for DJSS problems. The maximum size of the instruction set is 500 ($K = 500$). Each OAM can update up to 3 instructions ($M = 3$) with a maximum moving distance of 0.1 ($V = 0.1$). The instruction adding

rate in OAM is set as 0.65, which is the same as the adding rate in macro mutation. To highlight the effectiveness of FLC, the genetic operator rates in LCLGP is set as “swap : macro mutation : linear crossover : duplication : OAM = 0.1 : 0.2 : 0.2 : 0.1 : 0.4”. The micro mutation is appended after the first three genetic operators. Because of the shorter programs, the linear crossover in LCLGP swaps up to 15 instructions in SR problems and up to 10 instructions in DJSS problems.

In the SRbench experiments, we perform some fine-tuning of the LCLGP parameters. Specifically, each LCLGP individual for SRBench has up to 20 instructions and manipulates up to 10 registers. To make full use of the program space, we define 8 out of the 10 registers as output registers. The 8 output registers are appended by a linear regression method to sum up into an output scalar. Each OAM can update up to 2 instructions. The other parameters are kept the same as the ones in the first experiment.

All the compared methods share the same primitive set for SR and DJSS problems, respectively. In the SR problems, the compared methods use a function set of $\{+, -, \times, AQ, \sin, \cos, \sqrt{\cdot}, \ln(|\cdot|)\}$. $AQ(a, b)$ is the analytical quotient, returning $\frac{a}{\sqrt{1+b^2}}$. The terminal set consists of the problem input features. In the DJSS problems, the compared methods use a function set of $\{+, -, \times, \div, \max, \min\}$ and a terminal set with thirteen input features (e.g., the processing time of an operation and its next operation) [Huang et al. 2022b]. In the SRBench experiment, LCLGP additionally uses an ephemeral real constant, ranging from $[-5, 5]$, in its terminal set.

5 Results

5.1 Experiment I – Basic SR and DJSS Problems

5.1.1 Test Performance. Table 1 shows the mean and standard deviation of test performance in SR and DJSS problems over 50 independent runs. The best test performance among the compared methods is highlighted in bold font. We perform a Friedman test with a significance level of 0.05 and a Wilcoxon rank-sum test with a significance level of 0.05 and with Bonferroni correction to analyze the test performance. The $+$, $-$, and $\approx$ indicate the compared method is significantly better than, significantly worse than, and statistically similar with LCLGP based on the Wilcoxon rank-sum test. The mean ranks by the Friedman test show that the proposed LCLGP has a very competitive mean rank (1.54) with the state-of-the-art method FLO (1.85) and is significantly better than the basic LGP in overall, with a p-value of 0.018. In addition, the Wilcoxon test shows that LCLGP is significantly better than the basic LGP and very competitive with FLO in most of the problems, achieving 9 out of the 13 the best performance in the comparison. LCLGP also achieves the smallest standard deviation in the comparison on 8 out of the 13 problems over the 50 independent runs. Table 1 highlights the improvement from the basic LGP. Specifically, “Normalized Gap” shows the mean performance gap between LCLGP and the basic LGP, normalized by the standard deviation of the basic LGP. We can see that LCLGP has an improvement for more than 20% (see the “Normalized Gap” column) of the standard deviation in most of the problems. The results imply that the proposed FLC method effectively compresses FLs. Searching over the compressed FL is very similar to searching over the optimized FL by the FLO method.

5.1.2 Training Convergence. To verify the training efficiency of LCLGP, we investigate the training convergence of the compared methods in this section. Fig. 6 shows the training convergence of the compared methods on six example problems. We can see that the proposed method (i.e., orange curves) reaches better fitness than the baseline within the same number of generations, particularly on Nguyen4 and Airfoil. In addition, LCLGP has a very similar training efficiency to FLO, indicating that the compressed FLs successfully extract and retain effective solutions from the entire landscape. Although there is virtually no difference between LCLGP and FLO in terms of training and test performance, the real strength of FLC comes with its training time, as shown in the next subsection.

Table 1. Mean (standard deviation) of test RSE for SR problems and optimization objectives for test DJSS problems. “Normalized Gap” indicates the mean performance gap between LCLGP and the basic LGP, normalized by the standard deviation of the basic LGP.

Problems	Basic LGP	FLO	LCLGP	Normalized Gap
Symbolic Regression				
Nguyen4	0.018 (0.051)	≈ 0.022 (0.05)	0.002 (0.004)	-31%
Nguyen5	0.044 (0.273)	≈ 0.005 (0.008)	0.005 (0.007)	-14%
Keijzer11	0.213 (0.097)	≈ 0.162 (0.128)	0.184 (0.087)	-30%
R1	0.003 (0.004)	≈ 0.003 (0.005)	0.003 (0.003)	-22%
Airfoil	1.001 (0.764)	≈ 0.869 (0.401)	0.827 (0.284)	-23%
Bhouse	0.371 (0.092)	≈ 0.341 (0.092)	0.321 (0.05)	-53%
Redwine	0.686 (0.033)	≈ 0.686 (0.042)	0.685 (0.028)	-2%
Dynamic Job Shop Scheduling				
DJSS-Tmax	3999.2 (90.9)	≈ 3967.5 (88.3)	3939 (96.9)	-66%
DJSS-Tmean	1118.2 (10.7)	≈ 1115 (7.7)	1114.2 (9.8)	-38%
DJSS-WTmean	1729.6 (27.7)	≈ 1725 (25.1)	1733.6 (29.2)	14%
DJSS-Fmax	4585.4 (126.1)	≈ 4477.4 (71.4)	4509 (87.3)	-61%
DJSS-Fmean	1566.8 (10.8)	≈ 1567.4 (12.1)	1564.6 (9.5)	-21%
DJSS-WFmean	2715.8 (16.4)	≈ 2711.6 (26.4)	2725.5 (29.9)	59%
mean rank	2.62	1.85	1.54	
p-value (vs. LCLGP)	0.018	1		

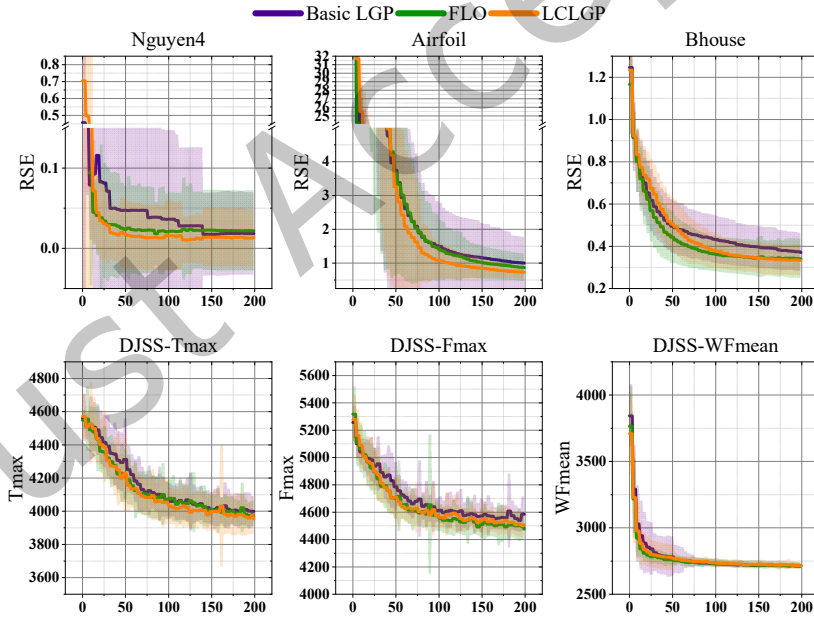


Fig. 6. Training convergence (shadow for standard deviation) on the six example problems. The X-axis is the generation.

5.1.3 Running Time. The compressed FLs substantially reduce the LGP training time. Table 2 shows the average training time and its standard deviation for the compared methods. The bold font highlights the best running

Table 2. Average training time (standard deviation) over the 50 runs in seconds

Problems	Basic LGP	FLO	LCLGP
Symbolic Regression			
Nguyen4	29.5 (1.2)	19564 (947.1)	22.8 (0.5)
Nguyen5	27.5 (1)	21044.3 (861.3)	20.7 (0.5)
Keijzer11	31.7 (1.2)	21322.5 (941.5)	29.3 (1.4)
R1	30.6 (1.2)	20266.8 (1082)	21.3 (0.4)
Airfoil	80.8 (3.5)	30748.5 (1386.5)	66.4 (1.9)
Bhouse	47.1 (1.7)	59189.1 (2565.8)	33.7 (0.7)
Redwine	89.4 (2.4)	57498.8 (1871.7)	65 (1.6)
Dynamic Job Shop Scheduling			
DJSS-Tmax	11043.6 (168.5)	70432.2 (1398.1)	10515.7 (291)
DJSS-Tmean	9612.3 (179.6)	64585.4 (1289.7)	7638.2 (122.6)
DJSS-WTmean	9758.9 (193.6)	67615.6 (1470.8)	8810.7 (212.1)
DJSS-Fmax	11480.3 (170.4)	64937.4 (1186)	10572.1 (178.2)
DJSS-Fmean	9521.8 (233.5)	67051.7 (1207.6)	7684.5 (180.1)
DJSS-WFmean	9601.4 (181)	65988.5 (1317.3)	9000 (228.6)

time. All experiments run on AMD EPYC 7702 computing nodes with a single thread, each thread having 16GB of RAM. LCLGP substantially reduced the training time of both basic LGP and FLO in all cases.

Specifically, LCLGP searches and optimizes on a compressed FL, saving a large amount of computation overhead to optimize and search the entire landscape. LCLGP reduces three orders of magnitude in SR problems and nearly one order of magnitude in DJSS problems from the state-of-the-art method (FLO).

In addition, LCLGP reduces the training time of basic LGP. This is because LCLGP uses high-level instructions, which integrates multiple functions into one instructions and reduces the overhead for manipulating registers from basic LGP. Basic LGP has to read and store the intermediate results from and to registers for every function. The test performance, training convergence, and running time verify the benefits of compressing FLs.

5.2 Experiment II – SRBench Problems

In the SRBench experiment, we compare LCLGP working with a linear regression model with more than ten popular or state-of-the-art regression methods. The test R-square, model size of output models, and training time are shown in Fig. 7. The results of other compared methods are provided on the SRBench website². The compared methods is ranked by the median test R^2 over the 122 datasets in a descending order. To compare the program size of LCLGP with other tree-based structures, the program size of LCLGP is estimated by the number of function nodes multiplied by a factor of 2.0. However, this is a rough estimation of LCLGP program size, as not all functions are binary functions (multiplied by a factor of 2.0).

We can see that LCLGP has a very competitive test R^2 with the state-of-the-art methods and ranks the second place among all the compared methods. In addition, the model size of LCLGP is competitive in the top-seven methods and is magnitudinally smaller than those of PSTree, SBP-GP, and XGB. LCLGP also has the shortest training time among the top seven methods, magnitudinally reducing the training time of PSTree and Operon. The results on SRBench highlight that FLC helps LGP achieve the state-of-the-art learning performance and significantly improve the training efficiency.

²<https://cavalab.org/srbench/results/>

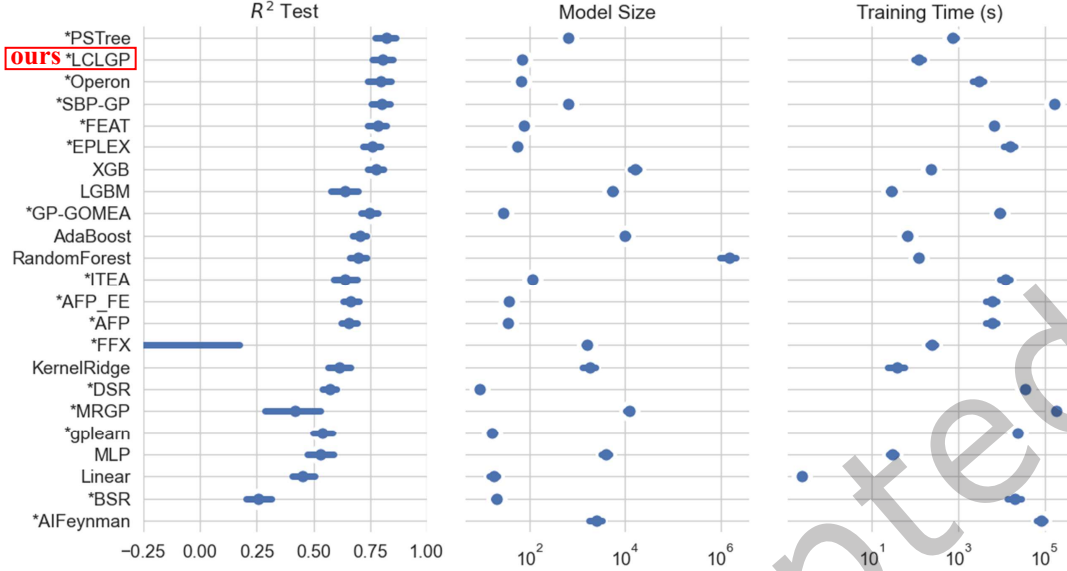


Fig. 7. Test R^2 , model size, and training time on SRBench 122 black-box regression problems.

6 Further Analysis

6.1 Example Compressed Landscape

To show the effectiveness of FLC intuitively, this section visualizes an example compressed landscape on a simple problem in Fig. 8. Because the visualization requires enumerating all the possible solutions in the search space, the simple problem has very limited primitives and programs with only 2 instructions. We select the Finger problem in [Huang et al. 2025] as an example. The primitive design follows [Huang et al. 2025]. Specifically, Finger is an SR problem with a ground truth formula of $f(x_0) = x_0 \times x_0 - x_0$ and a primitive set of $\{+, -, \times, R_0, x_0\}$.

LCLGP evolves a population of 50 individuals for 20 generations. LCLGP maintains an instruction set with a size of 20 ($K = 20$) and updates solutions by an OAM that changes at most one instruction each time ($M = 1$). Each LCLGP instruction has up to three functions.

Specifically, Fig. 8-A shows the initial and the optimized FLs by FLO [Huang et al. 2025]. In contrast, Fig. 8-C shows the compressed FLs from the initial to the final generation. Each node is a candidate 2-instruction program, whose color from red, to white, to blue indicates poor, mediate, and good fitnesses. Fig. 8-B shows the initial and the compressed set of instructions.

We have the following observations on Fig. 8:

- (1) For subfig. A., although the FLO optimizes the FL of Finger and aggregates good solutions to the bottom-left region [Huang et al. 2025], the good solutions are sparse, and there is only one global optimum.
- (2) Subfig. B. shows the instruction set before and after compression. The instruction set uses higher-level instructions (i.e., with more functions) to construct compressed FLs, so that the solutions on the compressed FLs have more diverse behaviors than raw FLs.
- (3) Subfig. C. shows the compressed FLs over evolution. We can see that there are more and more good solutions being included in the compressed FLs (i.e., more and more dark nodes).

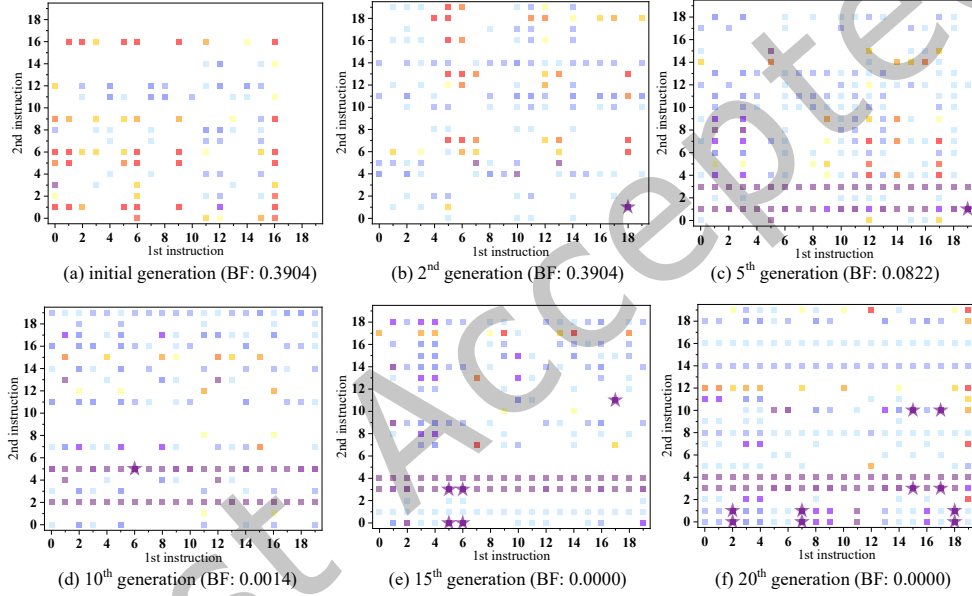
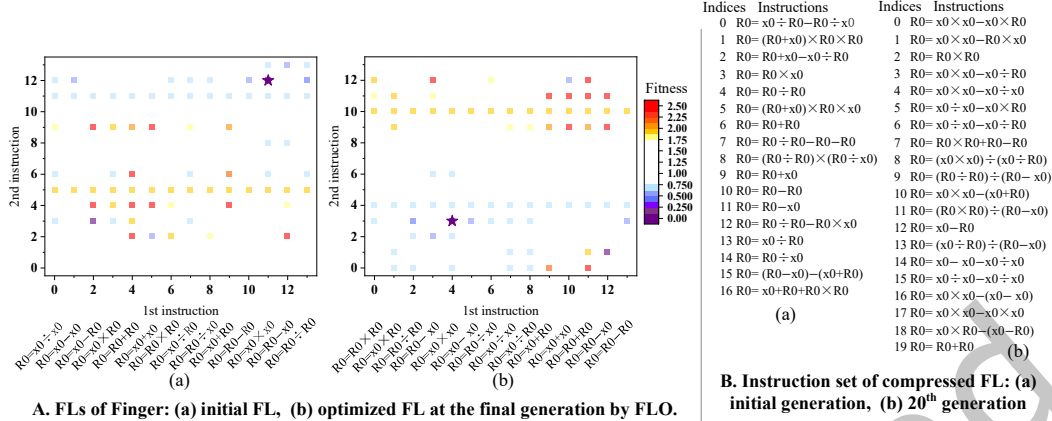


Fig. 8. Visualization of example compressed FLs of Finger. A: The original and the optimized FLs of Finger by FLO [Huang et al. 2025]. The color indicates the RSE values, red for poor fitness and blue for good fitness, as shown in the legend. The purple stars are the optima. B: The instruction set (i.e., instructions and their indices) before and after compression. C: The compressed FLs from the initial to the final generation. The node color has the same setting as the sub-figure A. “BF” indicates “the best fitness” so far over evolution.

- (4) Before LCLGP actually finds the optimal solution (BF=0.00) at the 15th generation, the compressed FL already includes the optimum in 2nd to 10th generations. Although the optimum is far away from the origin at the 2nd and 5th generations, we can see that the good solutions are denser and denser around the origin, and the optimum is moving toward the origin during evolution. It implies that the proposed FLC helps LGP filter out effective solutions from the raw search space and prioritize them around the origin.

This helps LGP find effective solutions in a shorter time if it moves toward the origin on the compressed FL.

- (5) In addition, because of the higher-level instructions, there are more global optima in the compressed FLs, making the compressed FLs a kind of zooming in on a promising search region.

6.2 Effect of Compressing FLs

This subsection investigates the effect of compressing FLs and the parameter robustness by varying the key parameters in LCLGP. If the proposed FLC method really compresses FLs, the following three hypotheses should hold.

- (1) Compressing FLs into too few or too many symbols does not improve the performance, either losing too much information or including too many redundant symbols.
- (2) Evolving higher and higher-level instructions is equivalent to compressing more and more symbols and information into one high-level symbol. This increases the chance of redundant and overlapping symbols in compressed FLs when we only have a limited number of sampled solutions from the entire search space. Thus, evolving too high-level instructions does not help LCLGP performance.
- (3) If the compressed FLs really filter out useful instructions from the raw search space, increasing the number of varying instructions should not degenerate LGP performance as much as basic LGP. Basic LGP is recommended to mutate only one instruction each time to reduce destructive moves (i.e., offspring is worse than parents).

To validate these three assumptions, we evaluate LCLGP on eleven SR problems, including the seven SR problems in Section 4.1 and four addition real-world regression problems (Tower [Zhong et al. 2016], CNN, Air quality, and Concrete). We apply a Friedman test with a significance level of 0.05 to analyze the test R^2 among different parameter settings.

Fig. 9 shows the mean ranks by the Friedman test when varying the key parameters in LCLGP over eleven SR problems. First, the overall performance of LCLGP shows a rough “V” over different instruction set sizes (K), indicating that including too few and too many instructions in the instruction set both reduce the LCLGP performance. This validates our first assumption. Particularly, $K = 10$ has a significantly worse performance than $K = 500$ with a p-value of 0.045, while the other settings of K are statistically similar based on the Friedman test. The result implies that reducing too much information from FLs is more harmful than including redundant symbols.

Second, the mean rank over the maximum number of functions in each instruction becomes better from 1 to 3 instructions, then grows from 3 to 15 instructions. Specifically, to ensure a similar maximum program size for fair comparison, we reduce the maximum number of instructions in LCLGP programs from 100, 30, 10, to 3 when increasing the maximum number of functions from 1, 3, 7, to 15. The results show that evolving higher-level instructions helps LCLGP to some extent, but too-high-level instructions lead to a large number of redundant instructions in the compressed FL, resulting in poor performance. This validates the second hypothesis.

Third, increasing the maximum number of varying instructions M in LCLGP shows a similar trend with the basic LGP of decreasing LGP performance [Brameier and Banzhaf 2007]. However, varying 3 and 5 instructions in LCLGP are still competitive (and even better than in some cases) with varying only 1 instruction. This is different from basic LGP, which gets worse results when increasing the number of varying instructions. This implies that by maintaining effective instructions in the compressed FL, it is safe and efficient for LCLGP to increase its variation step size, validating the third hypothesis. The mean ranks of the maximum number of functions in an instruction and the maximum number of varying instructions do not show a significant difference with each other based on the Friedman test, also implying the robustness of these two parameters. Overall, the validation of the three assumptions implies that the proposed FLC method successfully compresses FLs of LGP.

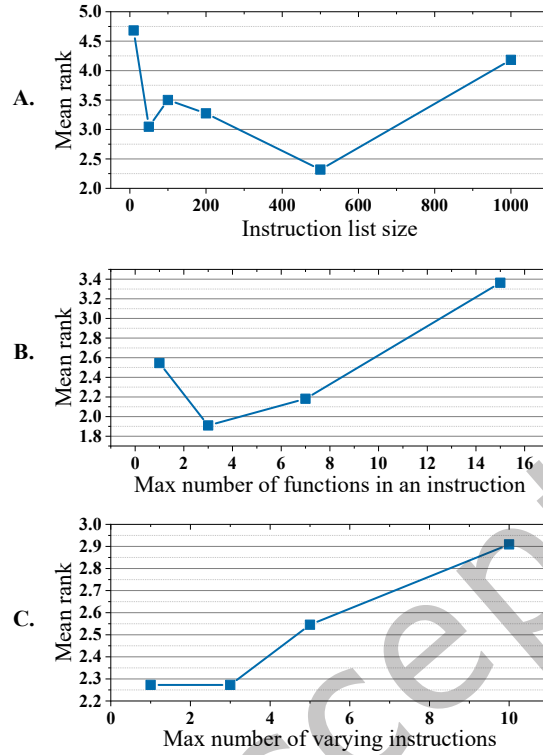


Fig. 9. Mean ranks for different parameter settings by the Friedman test. A. Instruction set size K ; B. Maximum number of functions in each instruction; C. Maximum number of varying instructions M .

6.3 Example Program

This section investigates the example output programs by LCLGP for solving Nguyen4. Fig. 10 shows the example programs by FLO and LCLGP. We select the example program with the best test performance and with the smallest program size over the 50 runs. Although the two programs can be eventually simplified as the ground truth formula of Nguyen4, $f(x) = x^6 + x^5 + x^4 + x^3 + x^2 + x$, the raw output program from FLO is much longer and more redundant than the one from LCLGP. This implies that LCLGP has a more compact representation than FLO by using high-level instructions.

7 Conclusions

This paper explicitly addressed fitness landscape compression by maintaining a dynamic list of effective symbols. We maintain the dynamic symbol list by adding symbols from competitive solutions, prioritizing symbols by fitness landscape optimization method, and removing less effective symbols. We implement this method based on a linear genetic programming method, which has a straightforward relationship with the fitness landscape. Based on the results of symbolic regression and dynamic job shop scheduling problems, we have the following conclusions:

FLO			LCLGP		
Initializing all registers as 1.0			Initializing all registers as 1.0		
Ins 0:	(R1= (AQ R1 R7))	(continue)	Ins 0:	(R2= (* R5 In0))	
//Ins 1:	(R4= (sin R1))	//Ins 39:	(R2= (+ R1 R0))	//Ins 1:	(R1= (* (* In0 R2) (* R7 In0)))
//Ins 2:	(R6= (sqr R1))	Ins 40:	(R1= (+ R6 R3))	Ins 2:	(R0= (sqr R7))
//Ins 3:	(R2= (AQ R3 R4))	Ins 41:	(R6= (sqr R3))	//Ins 3:	(R1= (* (* In0 R4) (* R0 In0)))
//Ins 4:	(R2= (+ R1 R0))	//Ins 42:	(R2= (AQ R3 R4))	//Ins 4:	(R1= (* (* R3 R4) (* R5 In0)))
Ins 5:	(R2= (+ R1 R0))	//Ins 43:	(R2= (+ R7 R0))	Ins 5:	(R1= (* (* In0 R2) (* R0 In0)))
//Ins 6:	(R0= (* R6 In0))	//Ins 44:	(R5= (AQ R6 R2))	//Ins 6:	(R0= (+ R3 R1))
//Ins 7:	(R5= (AQ R6 R2))	Ins 45:	(R0= (* R6 In0))	//Ins 7:	(R0= (+ R6 R1))
Ins 8:	(R0= (— R2 R1))	Ins 46:	(R6= (AQ R4 R7))	Ins 8:	(R5= (* R7 In0))
Ins 9:	(R0= (* In0 R0))	Ins 47:	(R2= (+ R1 R0))	//Ins 9:	(R0= (+ R3 R1))
Ins 10:	(R1= (+ R3 R0))	Ins 48:	(R5= (AQ R6 R2))	//Ins 10:	(R0= (* (+ In0 R0) (* R3 R1)))
Ins 11:	(R5= (AQ R3 R2))	Ins 49:	(R0= (— R2 R1))	Ins 11:	(R0= (+ R6 R1))
//Ins 12:	(R4= (— R3 R1))	Ins 50:	(R0= (* In0 R0))	Ins 12:	(R1= (* (* R3 R4) (* R5 In0)))
Ins 13:	(R2= (— R7 R7))	Ins 51:	(R1= (+ R0 R3))	Ins 13:	(R3= (+ R6 R1))
Ins 14:	(R0= (— R1 R7))	Ins 52:	(R7= (AQ R3 R2))	//Ins 14:	(R1= (* (* In0 R6) (* R0 R2)))
Ins 15:	(R0= (— R2 R3))	//Ins 53:	(R4= (— R3 R1))	//Ins 15:	(R6= (* In0 R1))
Ins 16:	(R1= (— R5 R6))	Ins 54:	(R2= (— R7 R7))	Ins 16:	(R0= (* (* In0 R0) (+ R3 In0)))
Ins 17:	(R2= (— R0 In0))	Ins 55:	(R6= (— R2 R3))		
Ins 18:	(R6= (* R6 R6))	Ins 56:	(R2= (— R0 In0))		
//Ins 19:	(R0= (* In0 R7))	Ins 57:	(R5= (— R5 R6))		
Ins 20:	(R7= (* R1 In0))	Ins 58:	(R6= (* R6 R6))		
Ins 21:	(R0= (* In0 R7))	//Ins 59:	(R0= (* R7 In0))		
Ins 22:	(R0= (+ R6 R0))	Ins 60:	(R7= (* In0 R1))		
Ins 23:	(R5= (— R5 R2))	Ins 61:	(R0= (* In0 R7))		
Ins 24:	(R5= (— R2 R5))	Ins 62:	(R0= (+ R0 R6))		
Ins 25:	(R6= (AQ R0 In0))	Ins 63:	(R5= (— R5 R2))		
Ins 26:	(R7= (cos R5))	Ins 64:	(R5= (— R2 R5))		
Ins 27:	(R0= (— R1 In0))	//Ins 65:	(R6= (AQ R0 In0))		
Ins 28:	(R1= (* In0 R0))	//Ins 66:	(R7= (cos R5))		
Ins 29:	(R2= (* In0 R6))	//Ins 67:	(R1= (* In0 R0))		
Ins 30:	(R4= (— R5 R2))	Ins 68:	(R4= (— R5 R2))		
Ins 31:	(R1= (+ R1 R4))	//Ins 69:	(R1= (+ R1 R4))		
Ins 32:	(R2= (+ R1 R0))	//Ins 70:	(R2= (+ R0 R1))		
//Ins 33:	(R5= (AQ R6 R2))	Ins 71:	(R1= (* R0 In0))		
//Ins 34:	(R0= (— R2 R1))	Ins 72:	(R1= (+ R1 R4))		
Ins 35:	(R2= (— R2 R7))	Ins 73:	(R2= (+ R1 R0))		
Ins 36:	(R4= (* R2 R7))	Ins 74:	(R2= (— R2 R4))		
Ins 37:	(R7= (— R7 R1))	Ins 75:	(R0= (* R2 In0))		
//Ins 38:	(R1= (+ R1 R4))				

$$f(x) = x^6 + x^5 + x^4 + x^3 + x^2 + x$$

Fig. 10. Example programs for regressing Nguyen4 by FLO (left) and LCLGP (right), respectively. “//”: the ineffective instructions (i.e., intron). R0~7: the eight registers. In0: the input x . “sqr”: $\sqrt{\cdot}$

- (1) The fitness landscape compression is beneficial for searching for effective symbolic solutions. The LGP method with fitness landscape compression comprehensively achieves competitive performance with state-of-the-art methods in terms of test performance, model size, and training convergence. In addition, the results on the two types of problems imply that the proposed method is general enough for a wide range of applications.
- (2) The fitness landscape compression improves the search efficiency. The proposed LCLGP substantially reduces the training time in both symbolic regression and dynamic job shop scheduling problems.
- (3) The fitness landscape compression successfully extracts effective and high-level symbols from the raw search space. Our further analyses show that the fitness landscape compression expands promising search regions and reduces the probability of destructive moves.

In the future, we have the following research directions. First, because of the usage of high-level instructions, programs evolved by LCLGP might not be concise enough. Searching for more concise programs is beneficial for interoperability and computation efficiency. Second, because the compressed FL might be deceptive (i.e., excluding optimal solutions), especially at the beginning of the search, improving the diversity of instructions to reduce the risk of deceptive FLs is an important direction.

References

- Baligh Al-Helali, Qi Chen, Bing Xue, and Mengjie Zhang. 2021. Multitree Genetic Programming with New Operators for Transfer Learning in Symbolic Regression with Incomplete Data. *IEEE Transactions on Evolutionary Computation* 25, 6 (2021), 1049–1063.
- Markus Brameier and Wolfgang Banzhaf. 2007. *Linear Genetic Programming*. Springer New York, NY.
- Qi Chen, Mengjie Zhang, and Bing Xue. 2017. Feature selection to improve generalization of genetic programming for high-dimensional symbolic regression. *IEEE Transactions on Evolutionary Computation* 21, 5 (2017), 792–806.
- F. O. De Franca, M. Virgolin, M. Kommenda, M. S. Majumder, M. Cranmer, G. Espada, L. Ingelse, A. Fonseca, M. Landajuela, B. Petersen, R. Glatt, N. Mundhenk, C. S. Lee, J. D. Hochhalter, D. L. Randall, P. Kamienny, H. Zhang, G. Dick, A. Simon, B. Burlacu, Jaan Kasak, Meera Machado, Casper Wilstrup, and W. G. La Cava. 2025. SRBench++: Principled Benchmarking of Symbolic Regression With Domain-Expert Interpretation. *IEEE Transactions on Evolutionary Computation* 29, 4 (2025), 1127–1137.
- Carlton Downey, Mengjie Zhang, and Will N. Browne. 2010. New crossover operators in linear genetic programming for multiclass object classification. In *Proceedings of the Annual Genetic and Evolutionary Computation Conference*. 885–892.
- Marko Durasevic, Domagoj Jakobovic, Marcella Scoczynski Ribeiro Martins, Stjepan Picek, and Markus Wagner. 2020. Fitness Landscape Analysis of Dimensionally-Aware Genetic Programming Featuring Feynman Equations. In *Proceedings of International Conference on Parallel Problem Solving from Nature*. 111–124.
- Matheus Campos Fernandes, Fabrício Olivetti De França, and Emilio Francesquini. 2023. HOTGP - Higher-Order Typed Genetic Programming. In *Proceedings of the Genetic and Evolutionary Computation Conference*. 1091–1099.
- Alcides Fonseca and Diogo Poças. 2023. Comparing the expressive power of Strongly-Typed and Grammar-Guided Genetic Programming. In *Proceedings of Genetic and Evolutionary Computation Conference*, Vol. 1. 1100–1108.
- Yifan He and Ferrante Neri. 2023. Fitness Landscape Analysis of Genetic Programming Search Spaces with Local Optima Networks. In *Proceedings of the Companion Conference on Genetic and Evolutionary Computation*. 2056–2063.
- Thomas Helmuth, Edward Pantridge, Grace Woolson, and Lee Spector. 2020. Genetic Source Sensitivity and Transfer Learning in Genetic Programming. In *Proceedings of the 2020 Conference on Artificial Life*. MIT Press, 303–311.
- Yuanzi Hong, Wei-Li Liu, Jinghui Zhong, Peng Liang, Jianhua Guo, and Chunying Li. 2024. Archive-based Cooperative Coevolution Genetic Programming for Workflow Scheduling. In *Proceedings of IEEE Conference on Artificial Intelligence*. 216–219.
- Ting Hu and Wolfgang Banzhaf. 2018. Neutrality, Robustness, and Evolvability in Genetic Programming. In *Genetic Programming Theory and Practice XIV*. 101–117.
- Ting Hu, Wolfgang Banzhaf, and Jason H. Moore. 2013. Robustness and evolvability of recombination in linear genetic programming. In *European Conference on Genetic Programming*, Vol. 7831. 97–108.
- Ting Hu, Joshua L. Payne, Wolfgang Banzhaf, and Jason H. Moore. 2011. Robustness, evolvability, and accessibility in linear genetic programming. In *European Conference on Genetic Programming*. 13–24.
- Ting Hu, Joshua L. Payne, Wolfgang Banzhaf, and Jason H. Moore. 2012. Evolutionary dynamics on multiple scales: A quantitative analysis of the interplay between genotype, phenotype, and fitness in linear genetic programming. *Genetic Programming and Evolvable Machines* 13, 3 (2012), 305–337.
- Zhixing Huang, Yi Mei, Fangfang Zhang, and Mengjie Zhang. 2022b. A Further Investigation to Improve Linear Genetic Programming in Dynamic Job Shop Scheduling. In *Proceedings of IEEE Symposium Series on Computational Intelligence*. 496–503.
- Zhixing Huang, Yi Mei, Fangfang Zhang, and Mengjie Zhang. 2024a. Multitask Linear Genetic Programming with Shared Individuals and its Application to Dynamic Job Shop Scheduling. *IEEE Transactions on Evolutionary Computation* 28, 6 (2024), 1546 – 1560.
- Zhixing Huang, Yi Mei, Fangfang Zhang, and Mengjie Zhang. 2024b. Toward Evolving Dispatching Rules With Flow Control Operations By Grammar-Guided Linear Genetic Programming. *IEEE Transactions on Evolutionary Computation* 29, 1 (2024), 217–231.
- Zhixing Huang, Yi Mei, Fangfang Zhang, Mengjie Zhang, and Wolfgang Banzhaf. 2025. Fitness Landscape Optimization Makes Stochastic Symbolic Search By Genetic Programming Easier. *IEEE Transactions on Evolutionary Computation* 29, 6 (2025), 2742 – 2756.
- Zhixing Huang, Yi Mei, and Jinghui Zhong. 2022a. Semantic Linear Genetic Programming for Symbolic Regression. *IEEE Transactions on Cybernetics* (2022), 1–14.
- Zhixing Huang, Jinghui Zhong, Liang Feng, Yi Mei, and Wentong Cai. 2020. A fast parallel genetic programming framework with adaptively weighted primitives for symbolic regression. *Soft Computing* 24, 10 (2020), 7523–7539.
- Ruwang Jiao, Bing Xue, and Mengjie Zhang. 2023. A Multiform Optimization Framework for Constrained Multiobjective Optimization. *IEEE Transactions on Cybernetics* 53, 8 (2023), 5165 – 5177.
- Ruwang Jiao, Bing Xue, and Mengjie Zhang. 2024. Learning to Preselection: A Filter-Based Performance Predictor for Multiobjective Feature Selection in Classification. *IEEE Transactions on Evolutionary Computation* (2024). Early access.
- Terry Jones and Stephanie Forrest. 1995. Fitness Distance Correlation as a Measure of Problem Difficulty for Genetic Algorithms. In *Proceedings of the 6th International Conference on Genetic Algorithms*. 184–192.
- K.E. Kinnear. 1994. Fitness landscapes and difficulty in genetic programming. In *Proceedings of the First IEEE Conference on Evolutionary Computation and IEEE World Congress on Computational Intelligence*. 142–147.

- John R Koza. 1994. *Genetic programming II: automatic discovery of reusable programs*. MIT press, MA, USA.
- J. R. Koza. 1995. Survey of genetic algorithms and genetic programming. In *Wescon conference record*. San Francisco, CA, USA, 589.
- Chao Lyu, Lijun Sun, and Yuhui Shi. 2021. Machine Learning based Fitness Landscape Smoothing for Guiding Evolutionary Optimization. In *Proceedings of International Conference on Computational Intelligence and Security*. IEEE, Chengdu, China, 6–10.
- Jun Ma, Haoran Shan, Fengyang Sun, Lin Wang, Shuangrong Liu, Houguan Zhu, Fenghui Gao, Junteng Zheng, Bo Yang, and Qinfei Li. 2025. NEEP-ADF: Neuro-encoded expression programming with automatically defined functions. *Information Sciences* 704 (June 2025), 121957.
- Yi Mei, Qi Chen, Andrew Lensen, Bing Xue, and Mengjie Zhang. 2023. Explainable Artificial Intelligence by Genetic Programming: A Survey. *IEEE Transactions on Evolutionary Computation* 27, 3 (2023), 621–641.
- Alberto Moraglio, Krzysztof Krawiec, and Colin G. Johnson. 2012. Geometric Semantic Genetic Programming. In *Proceedings of Parallel Problem Solving from Nature - PPSN XII*, Vol. 7491. Springer Berlin Heidelberg, Berlin, Heidelberg, 21–31.
- Luis Muñoz, Leonardo Trujillo, and Sara Silva. 2020. Transfer learning in constructive induction with Genetic Programming. *Genetic Programming and Evolvable Machines* 21, 4 (2020), 529–569.
- Miguel Nicolau and Alexandros Agapitos. 2021. Choosing function sets with better generalisation performance for symbolic regression models. *Genetic Programming and Evolvable Machines* 22, 1 (March 2021), 73–100.
- Marco S. Nobile, Paolo Cazzaniga, and Daniel A. Ashlock. 2019. Dilation Functions in Global Optimization. In *Proceedings of IEEE Congress on Evolutionary Computation*. IEEE, Wellington, New Zealand, 2300–2307.
- Daniele M. Papetti, Daniel A. Ashlock, Paolo Cazzaniga, Daniela Besozzi, and Marco S. Nobile. 2021. If You Can't Beat It, Squash It: Simplify Global Optimization by Evolving Dilation Functions. In *Proceedings of IEEE Congress on Evolutionary Computation*. IEEE, Kraków, Poland, 1414–1422.
- Daniele M. Papetti, Andrea Tangherloni, Davide Farinati, Paolo Cazzaniga, and Leonardo Vanneschi. 2023. Simplifying Fitness Landscapes Using Dilation Functions Evolved With Genetic Programming. *IEEE Computational Intelligence Magazine* 18, 1 (Feb. 2023), 22–31.
- Pierre Yves Passaggia, Asamaning Quansah, Nicolas Mazellier, Guy Y. Cornejo MacEda, and Azeddine Kourta. 2022. Real-time feedback stall control of an airfoil at large Reynolds numbers using linear genetic programming. *Physics of Fluids* 34, 4 (2022), 045108.
- Tomasz Praczyk and Maciej Szymkowiak. 2023. Using Linear Matrix Genetic Programming to Model Behavior of Underwater Vehicle.
- Darian Reyes Fernández De Bulnes, Allan De Lima, Aidan Murphy, Douglas Mota Dias, and Conor Ryan. 2024. Feature Encapsulation by Stages Using Grammatical Evolution. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. ACM, 531–534.
- Lino Rodríguez-Coayahuitl, Ansel Y. Rodríguez-González, Daniel Fajardo-Delgado, and Maria Guadalupe Sánchez Cervantes. 2025. Problem Decomposition Strategies and Credit Distribution Mechanisms in Modular Genetic Programming for Supervised Learning. *IEEE Transactions on Evolutionary Computation* 29, 6 (2025), 2785 – 2803.
- Xinming Shi, Leandro L. Minku, and Xin Yao. 2024. Evolving Memristive Reservoir. *IEEE Transactions on Neural Networks and Learning Systems* 35, 10 (2024), 13574–13588.
- Karel Slaný and Lukáš Sekanina. 2007. Fitness Landscape Analysis and Image Filter Evolution Using Functional-Level CGP. In *Proceedings of European Conference on Genetic Programming*. 311–320.
- Rebeka Čorić, Mateja Đumić, and Domagoj Jakobović. 2021. Genetic programming hyperheuristic parameter configuration using fitness landscape analysis. *Applied Intelligence* 51, 10 (2021), 7402–7426.
- Leonardo Vanneschi. 2004. *Theory and practice for efficient genetic programming*. phdthesis. Universit   de Lausanne, Facult   des sciences.
- Leonardo Vanneschi, Manuel Clergue, Philippe Collard, Marco Tomassini, and S  bastien V  rel. 2004. Fitness Clouds and Problem Hardness in Genetic Programming. In *Proceedings of Genetic and Evolutionary Computation Conference*. 690–701.
- Leonardo Vanneschi, Davide Farinati, Diogo Rasteiro, Liah Rosenfeld, Gloria Pietropoli, and Sara Silva. 2025. Exploring Non-bloating Geometric Semantic Genetic Programming. In *Genetic Programming Theory and Practice XXI*, Stephan M. Winkler, Wolfgang Banzhaf, Ting Hu, and Alexander Lalejini (Eds.). Springer Nature Singapore, Singapore, 237–258.
- L. Vanneschi and M. Tomassini. 2003. Pros and Cons of Fitness Distance Correlation in Genetic Programming. In *Proceedings of Genetic and Evolutionary Computation Conference Workshop Program*. 284–287.
- Leonardo Vanneschi, Marco Tomassini, Philippe Collard, and Manuel Clergue. 2003. Fitness Distance Correlation in Structural Mutation Genetic Programming. In *Proceedings of European Conference on Genetic Programming*. 455–464.
- Leonardo Vanneschi, Marco Tomassini, Philippe Collard, and S  bastien V  rel. 2006. Negative Slope Coefficient: A Measure to Characterize Genetic Programming Fitness Landscapes. In *Proceedings of European Conference on Genetic Programming*. 178–189.
- Gang Wang and Terence Soule. 2004. How to Choose Appropriate Function Sets for Gentic Programming. In *Genetic Programming*. Vol. 3003. Berlin, Heidelberg, 198–207.
- P.A. Whigham. 1995. Grammatically-based Genetic Programming. In *Proceedings of the Workshop on Genetic Programming: From Theory to Real-World Applications*. 33–41.
- Meng Xu, Fangfang Zhang, Yi Mei, and Mengjie Zhang. 2021. Genetic Programming with Archive for Dynamic Flexible Job Shop Scheduling. In *Proceedings of IEEE Congress on Evolutionary Computation*. Krak  w, Poland, 2117–2124.
- Fangfang Zhang, Yi Mei, Su Nguyen, and Mengjie Zhang. 2021a. Evolving Scheduling Heuristics via Genetic Programming With Feature Selection in Dynamic Flexible Job-Shop Scheduling. *IEEE Transactions on Cybernetics* 51, 4 (2021), 1797–1811.

- Fangfang Zhang, Yi Mei, Su Nguyen, and Mengjie Zhang. 2021b. Evolving Scheduling Heuristics via Genetic Programming with Feature Selection in Dynamic Flexible Job-Shop Scheduling. *IEEE Transactions on Cybernetics* 51, 4 (2021), 1797–1811.
- Fangfang Zhang, Yi Mei, Su Nguyen, and Mengjie Zhang. 2024. Survey on Genetic Programming and Machine Learning Techniques for Heuristic Design in Job Shop Scheduling. *IEEE Transactions on Evolutionary Computation* 28, 1 (2024), 147–167.
- Fangfang Zhang, Gaofeng Shi, Yi Mei, and Mengjie Zhang. 2025. Multiobjective Dynamic Flexible Job Shop Scheduling With Biased Objectives via Multitask Genetic Programming. *IEEE Transactions on Artificial Intelligence* 6, 1 (Jan. 2025), 169–183.
- Jinghui Zhong, Junlan Dong, Wei-Li Liu, Liang Feng, and Jun Zhang. 2025. Multiform Genetic Programming Framework for Symbolic Regression Problems. *IEEE Transactions on Evolutionary Computation* 29, 2 (2025), 429–443.
- Jinghui Zhong, Yew-Soon Ong, and Wentong Cai. 2016. Self-Learning Gene Expression Programming. *IEEE Transactions on Evolutionary Computation* 20, 1 (2016), 65–80.

Just Accepted